

LAPORAN PRATIKUM
ALGORITMA DAN PEMROGRAMAN

Double Linked List (DLL)

Disusun Oleh:

Khairun Nisa

2511532015

Dosen Pengampu:

DR. WAHYUDI, S.T, M.T

Asisten Pratikum:

Sherly Sukmadira



FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kepada Tuhan Yang Maha Esa atas terselesaikannya laporan praktikum Algoritma dan Pemrograman ini. Laporan ini disusun sebagai bentuk dokumentasi dan pertanggungjawaban atas pelaksanaan praktikum yang telah dilaksanakan pada tanggal 11 Mei 2026. Melalui laporan ini, penulis berharap dapat memberikan gambaran yang jelas mengenai Double Linked List (DLL).

Penulis mengucapkan terima kasih kepada Bapak Dosen serta asisten laboratorium yang telah membimbing selama praktikum berlangsung. Penulis menyadari sepenuhnya bahwa laporan ini masih jauh dari kata sempurna, baik dari segi penyusunan, bahasa, maupun kedalaman materi. Oleh karena itu, penulis dengan rendah hati menerima segala kritik dan saran yang membangun demi perbaikan dan penyempurnaan laporan di masa mendatang. Semoga laporan ini dapat bermanfaat bagi pembaca dan menjadi referensi untuk perkembangan ilmu pengetahuan di teknologi informasi.

Padang, 12 Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Pratikum	1
1.3 Manfaat Pratikum	2
BAB II PEMBAHASAN	3
2.1 NodeDLL.....	3
2.2 InsertDLL	3
2.3 PenelusuranDLL	7
2.4 HapusDLL	9
BAB III KESIMPULAN	13
3.1 Kesimpulan	13
DAFTAR PUSTAKA	15

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktru data merupakan salah satu dasar penting dalam pemrograman karena menentukan bagaimana data diorganisir, disimpan, dan diakses secara efesien. Dalam perkemabangannya, struktur data linier seperti linked list sering menjadi materi awal yang dipelajari sebelum beralih ke sturktur yang kompleks. Salah satu varian yang cukup sering digunakan adalah Double Linked List (DLL).

Doubly Linked List (DLL) merupakan struktur data yang terdiri dari node-node yang terhubung secara berpasangan, di mana setiap node memiliki pointer ke node sebelumnya (prev) dan node berikutnya (next). DLL menawarkan beberapa keunggulan dibandingkan dengan Single Linked List (SLL), terutama dalam hal operasi insertion dan deletion. Operasi insertion pada DLL dapat dilakukan lebih efisien di berbagai posisi, baik di awal, tengah, maupun akhir list. Hal ini karena DLL memungkinkan akses data dari dua arah, sehingga memungkinkan untuk menyisipkan node baru tanpa harus melalui seluruh list [1].

1.2 Tujuan Praktikum

1. Memahami konsep dasar dan cara kerja Double Linked List serta perbedaannya dengan struktru data linier sejenis lainnya seperti Single Linked List (SLL).
2. Mengimplementasikan kelas dan operasi utama DLL dalam bahasa pemrograman.
3. Menganalisis mekanisme manipulasi penunjuk prev dan next saat terjadi operasi tambah, baca, dan hapus, termasuk penanganan kasus tepi.
4. Melatih kemampuan debugging dan evaluasi kompleksitas waktu pada operasi dinamis.

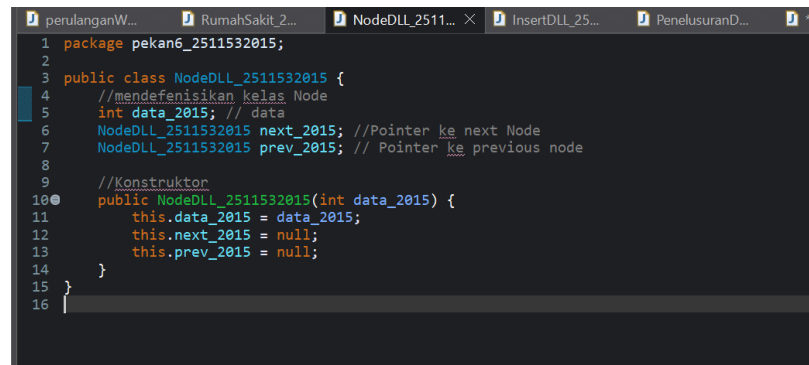
1.3 Manfaat Praktikum

1. Memberikan pengalaman langsung dalam menerjemahkan teori struktur data ke dalam kode yang berjalan, sehingga pemahaman konseptual menjadi lebih konkret dan tidak hanya bersifat hapalan.
2. Meningkatkan ketelitian logika pemrograman.
3. Menjadi fondasi yang kuat untuk mempelajari struktur data yang lebih kompleks di modul berikutnya.
4. Membiasakan mahasiswa dalam mempertimbangkan kelebihan dan keterbatasan suatu struktur data, sehingga kelak dapat memilih implementasi yang paling efisien sesuai dengan karakteristik masalah yang dihadapi.

BAB II

PEMBAHASAN

2.1 NodeDLL_2511532015



```
1 package pekan6_2511532015;
2
3 public class NodeDLL_2511532015 {
4     //mendefinisikan kelas Node
5     int data_2015; // data
6     NodeDLL_2511532015 next_2015; //Pointer ke next Node
7     NodeDLL_2511532015 prev_2015; // Pointer ke previous node
8
9     //Konstruktor
10    public NodeDLL_2511532015(int data_2015) {
11        this.data_2015 = data_2015;
12        this.next_2015 = null;
13        this.prev_2015 = null;
14    }
15 }
16 }
```

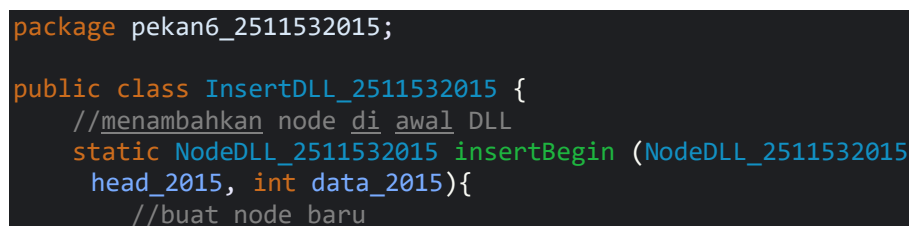
Gambar 2.1.1: Kode Program NodeDLL_2511532015

Kelas ini merupakan komponen fundamental dalam implementasi struktur data Double Linked List (DLL). Kelas ini berfungsi sebagai blueprint untuk setiap node yang akan disimpan dalam daftar terhubung. Kelas ini dideklarasikan public yang menandakan bahwa kelas ini dapat diakses kelas lain. Terdiri dari 3 atribut utama:

- data_2015: menyimpan nilai actual yang dibawa oleh node.
- next_2015: referensi yang menunjuk ke node berikutnya dalam urutan daftar.
- prev_2015: referensi yang menunjuk ke node sebelumnya. Atribut inilah yang memungkinkan traversal dua arah dan mempermudah operasi penghapusan atau penyisipan di posisi tengah maupun akhir tanpa perlu menelusuri dari awal.

Pada konstruktor variable next_2015 dan prev_2015 diinisialisasikan null karena node yang baru diciptakan belum terhubung dengan node lain.

2.2 InsertDLL_2511532015



```
package pekan6_2511532015;

public class InsertDLL_2511532015 {
    //menambahkan node di awal DLL
    static NodeDLL_2511532015 insertBegin (NodeDLL_2511532015
        head_2015, int data_2015){
        //buat node baru
```

```

        NodeDLL_2511532015 new_node = new NodeDLL_2511532015
(data_2015);
        //jadikan pinter nextnya head
        new_node.next_2015 = head_2015;
        //jadikan pointer prev head ke new node
        if (head_2015 != null){
            head_2015.prev_2015 = new_node;
        }
        return new_node;
    }
    //fungsi menambahkan node di akhir
    public static NodeDLL_2511532015 insertEnd(NodeDLL_2511532015
head_2015, int newData){
        //buat node baru
        NodeDLL_2511532015 newNode = new NodeDLL_2511532015
(newData);
        //jika dll null jadikan head
        if (head_2015 == null){
            head_2015 = newNode;
        }
        else {
            NodeDLL_2511532015 curr_2015 = head_2015;
            while (curr_2015.next_2015 != null){
                curr_2015 = curr_2015.next_2015;
            }
            curr_2015.next_2015 = newNode;
            newNode.prev_2015 = curr_2015;
        }
        return head_2015;
    }
    //fungsi menambahkan node di posisi terbatas
    public static NodeDLL_2511532015 insertAtPosition
(NodeDLL_2511532015 head_2015, int pos_2015, int new_data) {
        //buat node baru
        NodeDLL_2511532015 new_node = new NodeDLL_2511532015
(new_data);
        if (pos_2015 == 1) {
            new_node.next_2015 = head_2015;
            if (head_2015 != null) {
                head_2015.prev_2015 = new_node;
            }
            head_2015 = new_node;
            return head_2015;
        }
        NodeDLL_2511532015 curr_2015 = head_2015;
        for (int i = 1; i < pos_2015 - 1 && curr_2015 != null; ++i)
        {
            curr_2015 = curr_2015.next_2015;
        }
        if (curr_2015 == null) {
            System.out.println ("Posisi tidak ada");
            return head_2015;
        }
        new_node.prev_2015 = curr_2015;
        new_node.next_2015 = curr_2015.next_2015;
        curr_2015.next_2015 = new_node;
        if (new_node.next_2015 != null) {

```

```

        new_node.next_2015.prev_2015 = new_node;
    }
    return head_2015;
}
public static void printList (NodeDLL_2511532015 head_2015) {
    NodeDLL_2511532015 curr_2015 = head_2015;
    while (curr_2015 != null) {
        System.out.print (curr_2015.data_2015 + " <-> ");
        curr_2015 = curr_2015.next_2015;
    }
    System.out.println ();
}
public static void main (String[] args) {
    //membuat dll 2 <-> 3 <-> 5
    NodeDLL_2511532015 head_2015 = new NodeDLL_2511532015 (2);
    head_2015.next_2015 = new NodeDLL_2511532015 (3);
    head_2015.next_2015.prev_2015 = head_2015;
    head_2015.next_2015.next_2015 = new NodeDLL_2511532015 (5);
    head_2015.next_2015.next_2015.prev_2015 =
    head_2015.next_2015;
    //cetak DLL awal
    System.out.print ("DLL Awal: ");
    printList (head_2015);
    //tambah 1 di awal
    head_2015 = insertBegin (head_2015, 1);
    System.out.print("Simpul 1 ditambah di awal: ");
    printList(head_2015);
    //tambah 6 di akhir
    System.out.print ("Simpul 6 ditambah di akhir: ");
    int data_2015 = 6;
    head_2015 = insertEnd (head_2015, data_2015);
    printList (head_2015);
    //menambahkan node 4 di posisi 4
    System.out.print ("Tambah node 4 di posisi 4: ");
    int data2_2015 = 4;
    int pos_2015 = 4;
    head_2015 = insertAtPosition (head_2015, pos_2015,
    data2_2015);
    printList (head_2015);
}
}

```

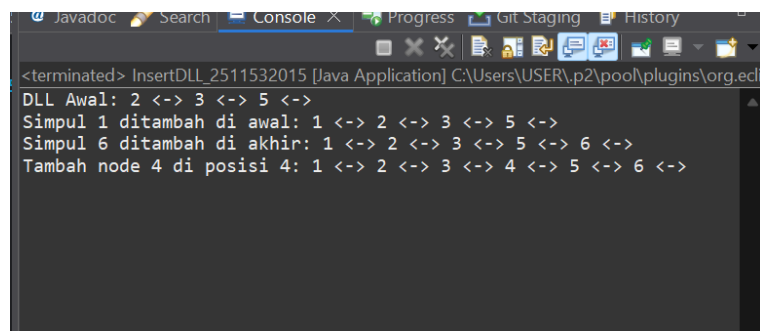
Kelas ini menambahkan elemen baru ke dalam terhubung, baik di posisi awal, akhir, dan posisi tertentu di tengah daftar. Kelas ini mempunyai beberapa method:

- Method insertBegin (menambahkan di awal daftar): pertama membuat node baru dengan data yang diterima sebagai parameter kemudian menetapkan pointer next_2015 node bar uke head_2015 (node pertama saat ini). Jika daftar tidak kosong (head_2015!= null)

pointer prev_2015 dari head_2015 lama diarahkan kembali ke node baru lalu retron new_node.

- Method insertEnd (menambahkan di akhir): membuat node baru dengan data yang diterima sebagai parameter. Jika head_2015 == null head_2015 akan menjadi node yang baru. Jika false, dilakukan traversal (penelusuran) menggunakan variable curr_2015 hingga mencapai node terakhir.
- Method insertAtPosition (menambahkan di posisi tertentu): jika poss_2015 == 1 langsung didelegasikan ke logika insertBegin. Untuk penelusuran menggunakan loop for untuk berpindah ke node tepat sebelum posisi tare=get (pos_2015 - 1). Jika posisi melebihi pajang daftar, ptogram mencetak [esan error dan mengembalikan head tanpa perubahan.
- Method printList (menampilkan isi daftar):method ini melakukan traversal dari head_2015 hingga akhir daftar, mencetak setiap data diikuti symbol <-> untuk memvisualisasikan sifat dua arah dari DLL.
- Main method (pengujian method-method di atas): pertama menambahkan manual daftar awal 2, 3, 5 terus memanggil ketiga method secara berurutan untuk memvalidasi kebenaran operasi mulai dari insertBegin, insertEnd, hingga insertAtPosition.

Program ini ketika dijalankan akan menghasilkan output:



```
<terminated> InsertDLL_2511532015 [Java Application] C:\Users\USER\p2\pool\plugins\org.ecl
DLL Awal: 2 <-> 3 <-> 5 <->
Simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
Simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
Tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->
```

Gambar 2.2.1: Output kode program InsertDLL_2511532015

Output ini membuktikan bahwa operasinya berjalan dengan lancar, mulai dari daftar awal yang hanya tersedia 2, 3, 5. Kemudian method insertBegin

dipanggil dengan menambahkan angka 1 di awal, selanjutnya method insertEnd dipanggil dengan menambahkan angka 6 di akhir, terakhir method insertAtPosition dipanggil dengan menambahkan angka 4 di posisi ke 4.

2.3 PenelusuranDLL_2511532015

```
package pekan6_2511532015;

public class PenelusuranDLL_2511532015 {
    //fungsi penelusuran maju
    static void forwardTraversal (NodeDLL_2511532015 head_2015)
    {
        //memulai penelusuran dari head
        NodeDLL_2511532015 curr_2015 = head_2015;
        //lanjutkan sampai akhir
        while (curr_2015 != null) {
            //print data
            System.out.print (curr_2015.data_2015 + " <-> ");
            //pindahkan ke node berikutnya
            curr_2015 = curr_2015.next_2015;
        }
        //print spasi
        System.out.println ();
    }
    //fungsi penelusuran mundur
    static void backwardTraversal (NodeDLL_2511532015
    tail_2015) {
        //mulai dari akhir
        NodeDLL_2511532015 curr_2015 = tail_2015;
        //lanjut sampai head
        while (curr_2015 != null) {
            //cetak data
            System.out.print (curr_2015.data_2015 + " <-> ");
            //pindahkan ke node sebelumnya
            curr_2015 = curr_2015.prev_2015;
        }
        //cetak spasi
        System.out.println();
    }
    public static void main (String[] args) {
        //cetak dll
        NodeDLL_2511532015 head_2015 = new NodeDLL_2511532015 (1);
        NodeDLL_2511532015 second_2015 = new NodeDLL_2511532015
        (2);
        NodeDLL_2511532015 third_2015 = new NodeDLL_2511532015 (3);

        head_2015.next_2015 = second_2015;
        second_2015.prev_2015 = head_2015;
        second_2015.next_2015 = third_2015;
        third_2015.prev_2015 = second_2015;

        System.out.println ("Penelusuran maju: ");
        forwardTraversal (head_2015);
    }
}
```

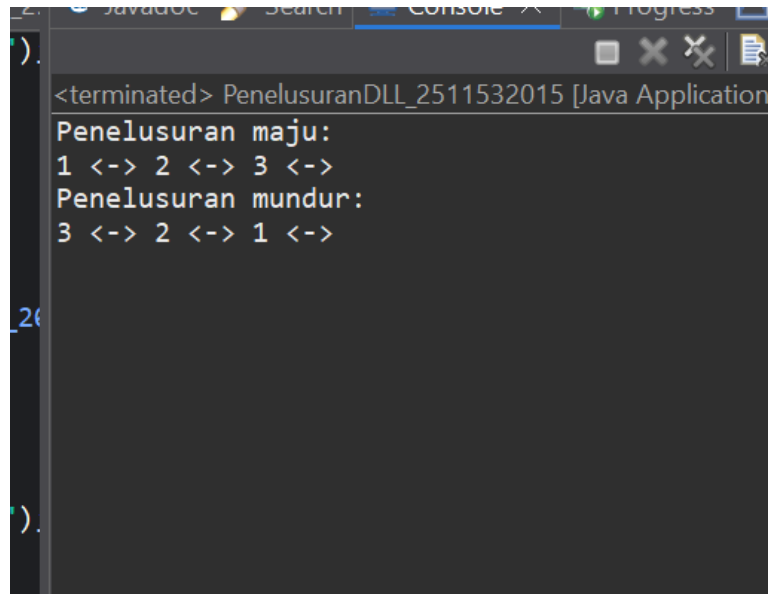
```

        System.out.println ("Penelusuran mundur: ");
        backwardTraversal (third_2015);
    }
}

```

Kelas ini bertujuan untuk pembacaan dan penelusuran (traversal) pada struktur data DLL. Struktur data DLL ini memiliki kemampuan untuk menelusuri data dua arah, yaitu maju (forward) dari kepala (head) ke ekor (tail), dan mundur (backward) dari ekor ke kepala. Kelas ini mengimplementasikan kedua kemampuan tersebut melalui dua metode statis.

- Method forwardTraversal (penelusuran maju): variable curr_2015 diinisialisasikan merujuk ke head_2015 sebagai titik awal penelusuran. Perulangan while akan terus berjalan selama curr_2015 tidak bernilai null, nilai null menandakan bahwa pointer telah mencapai akhir dari daftar. Pada setiap iterasi, data yang tersimpan dalam node saat ini (curr_2015.data_2015) dicetak ke layar. Pointer digeser ke node berikutnya dengan perintah curr_2015 = curr_2015.next_2015.
- Method backwardTraversal (penelusuran mundur): pointer bantu curr_2015 diarahkan ke tail_2015 (node terakhir). Perulangan while berjalan selama curr_2015 != null. Data pada node saat ini dicetak ke layar. Loop berhenti ketika pointer telah melewati node pertama (karena head.prev bernilai null).
- Main method: program membuat tiga objek terpisah bertipe NNodeDLL yang terdiri dari 1, 2, 3 tetapi pointer next dan prev mereka masih bernilai null (kosong) karena belum saling terhubung. Kemudian melakukan penghubungan node 1 dan node 2, kemudian node 2 dan node 3. Pengujian memanggil method forwardTraversal dan backwardTraversal untuk memastikan method berjalan dengan baik. Program akan menghasilkan output:



Gambar 2.3.1: Hasil output kode program PenelusuranDLL_2511532015

2.4 HapusDLL_2511532015

```
package pekan6_2511532015;

public class HapusDLL_2511532015 {
    //fungsi menghapus node di awal
    public static NodeDLL_2511532015 delHead (NodeDLL_2511532015
head_2015) {
        if (head_2015 == null) {
            return null;
        }
        NodeDLL_2511532015 temp_2015 = head_2015;
        head_2015 = head_2015.next_2015;
        if (head_2015 != null) {
            head_2015.prev_2015 = null;
        }
        return head_2015;
    }
    //fungsi menghapus di akhir
    public static NodeDLL_2511532015 dellLast (NodeDLL_2511532015
head_2015) {
        if (head_2015 == null) {
            return null;
        }
        if (head_2015.next_2015 == null) {
            return null;
        }
        NodeDLL_2511532015 curr_2015 = head_2015;
        while (curr_2015.next_2015 != null) {
            curr_2015 = curr_2015.next_2015;
        }
        //update pointer previous node
        if (curr_2015.prev_2015 != null) {
            curr_2015.prev_2015.next_2015 = null;
        }
    }
}
```

```

    }
    return head_2015;
}
//fungsi menghapus node posisi tertentu
public static NodeDLL_2511532015 delPos (NodeDLL_2511532015
head_2015, int pos_2015) {
    //jika DLL kosong
    if (head_2015 == null) {
        return head_2015;
    }
    NodeDLL_2511532015 curr_2015 = head_2015;
    //telusuri sampai ke node yang akan dihapus
    for (int i = 1; curr_2015 != null && i < pos_2015; ++i) {
        curr_2015 = curr_2015.next_2015;
    }
    //jika posisi tidak ditemukan
    if (curr_2015 == null) {
        return head_2015;
    }
    //update pointer
    if (curr_2015.prev_2015 != null) {
        curr_2015.prev_2015.next_2015 = curr_2015.next_2015;
    }
    if (curr_2015.next_2015 != null) {
        curr_2015.next_2015.prev_2015 = curr_2015.prev_2015;
    }
    //jika yang dihapus head
    if (head_2015 == curr_2015) {
        head_2015 = curr_2015.next_2015;
    }
    return head_2015;
}
//fungsi mencetak DLL
public static void printList (NodeDLL_2511532015 head_2015)
{
    NodeDLL_2511532015 curr_2015 = head_2015;
    while (curr_2015 != null) {
        System.out.print ( curr_2015.data_2015 + " ");
        curr_2015 = curr_2015.next_2015;
    }
    System.out.println ();
}
public static void main (String[] args) {
    //buat sebuah DLL
    NodeDLL_2511532015 head_2015 = new NodeDLL_2511532015 (1);
    head_2015.next_2015 = new NodeDLL_2511532015 (2);
    head_2015.next_2015.prev_2015 = head_2015;
    head_2015.next_2015.next_2015 = new NodeDLL_2511532015 (3);
    head_2015.next_2015.next_2015.prev_2015 =
head_2015.next_2015;
    head_2015.next_2015.next_2015.next_2015 = new
NodeDLL_2511532015 (4);
    head_2015.next_2015.next_2015.next_2015.prev_2015 =
head_2015.next_2015;
    head_2015.next_2015.next_2015.next_2015.next_2015 = new
NodeDLL_2511532015 (5);
}

```

```

        head_2015.next_2015.next_2015.next_2015.next_2015.prev_2015
        = head_2015.next_2015.next_2015.next_2015;

        System.out.print ("DLL Awal: ");
        printList(head_2015);

        System.out.print ("Setelah head dihapus: ");
        head_2015 = delHead (head_2015);
        printList (head_2015);

        System.out.print("Setelah node terakhir dihapus: ");
        head_2015 = delLast (head_2015);
        printList (head_2015);

        System.out.print ("Menga=hapus node ke 2: ");
        head_2015 = delPos (head_2015, 2);
        printList (head_2015);
    }
}

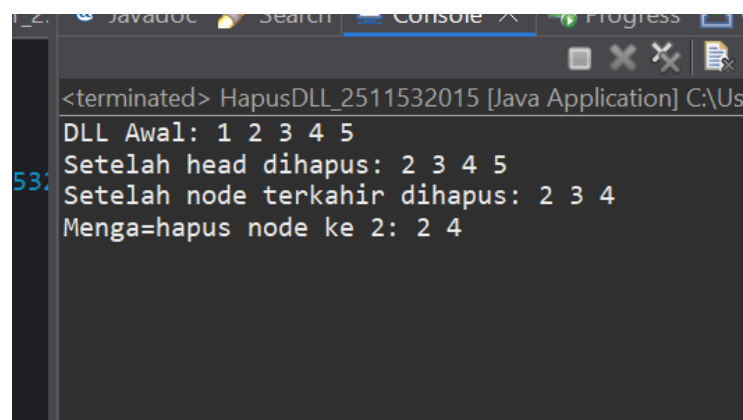
```

Kelas ini merupakan operasi penghapusan (deletion) pada struktru data DLL. Operasi penghapusan pada DLL lebih kompleks dibanding SLL karena setiap penghapusan node mengharuskan pemabruan dua arah pointer, pointer next_2015 dari node sebelumnya dan pointer prev_2015 dari node setelahnya.

- delHead (penghapusan di awal): program memeriksa apakah head_2015 == null, jika ya berarti daftar kosong dan tidak dapat dilakukan operasi. Variable temp_2015 digunakan untuk menyimpan referensi ke node head lama. Pointer head_2015 digeser ke node berikutnya melalui perintah head_2015 = head_2015.next_2015 yang membuat node kedua dalam daftar kini menjadi kepala baru. Jika head baru tidak bernilai null pointer prev_2015 pada head baru di set menjadi null.
- delLast (penghapusan di akhir): hanya bisa dijalankan ketika head_2015 tidak bernilai null. Jika head_2015.next_2015 bernilai null, berarti daftar hanya memiliki satu node. Variabel bantu curr_2015 diinisialisasi ke head_2015. Perulangan while dijalankan untuk menggeser curr_2015 ke node berikutnya hingga ditemukan node dengan next_2015 == null, yang menandakan node terakhir. Setelah node terakhir ditemukan, pointer next_2015 pada node

sebelumnya (`curr_2015.prev_2015`) di-set menjadi null. Ini memutuskan tautan ke node yang akan dihapus.

- `delPos` (penghapusan node pada posisi tertentu): Variabel bantu `curr_2015` digunakan untuk menelusuri daftar menggunakan perulangan `for` hingga mencapai node pada posisi yang diminta (`pos_2015`). Jika `curr_2015` bernilai null setelah penelusuran, berarti posisi yang diminta melebihi panjang daftar. Metode mengembalikan `head_2015` tanpa melakukan penghapusan. Jika node target memiliki node sebelumnya (`prev_2015 != null`), pointer `next_2015` pada node sebelumnya diarahkan ke node sesudah target (`curr_2015.next_2015`). Jika node target memiliki node sesudahnya (`next_2015 != null`), pointer `prev_2015` pada node sesudahnya diarahkan ke node sebelum target (`curr_2015.prev_2015`). Jika node yang dihapus adalah head (yaitu ketika `head_2015 == curr_2015`), maka `head_2015` diperbarui untuk menunjuk ke node berikutnya.
- `printList` (menampilkan isi daftar): berfungsi menampilkan seluruh elemen dalam DLL secara berurutan dari awal hingga akhir.
- Main method: program membangun daftar 1, 2, 3, 4, 5, secara manual menggunakan objek `NodeDLL` dan menghubungkan pointer `next_2015` serta `prev_2015` secara dua arah untuk setiap pasangan yang berdekatan. Dan dilakukan pengujian method penghapusan yang ada. Program ini ketika dijalankan akan menghasilkan output.



```
<terminated> HapusDLL_2511532015 [Java Application] C:\Us
DLL Awal: 1 2 3 4 5
Setelah head dihapus: 2 3 4 5
Setelah node terkahir dihapus: 2 3 4
Menga=hapus node ke 2: 2 4
```

Gambar 2.1.4: Hasil output kode program HapusDLL_2511532015

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan rangkaian praktikum struktur data mengenai Doubly Linked List (DLL) yang telah dilaksanakan, dapat disimpulkan beberapa hal berikut:

1. Konsep Dasar DLL: Doubly Linked List merupakan struktur data linier dinamis yang setiap nodenya terdiri dari tiga bagian, yaitu data, pointer next (menuju node sesudahnya), dan pointer prev (menuju node sebelumnya). Keberadaan dua pointer ini memungkinkan penelusuran data dilakukan secara dua arah, baik dari depan ke belakang maupun sebaliknya.
2. Implementasi Operasi Dasar: Melalui empat kelas yang dibuat (NodeDLL, InsertDLL, PenelusuranDLL, dan HapusDLL), kami berhasil menerapkan operasi fundamental pada DLL. Mulai dari pembuatan node, penyisipan data di posisi awal/akhir/tertentu, penelusuran dua arah, hingga penghapusan node dengan penyambungan ulang pointer agar rantai data tidak putus.
3. Krusialnya Manajemen Pointer: Inti dari keberhasilan program DLL terletak pada ketelitian mengatur pointer prev dan next. Saat melakukan insert atau delete, kedua pointer harus di-update secara bersamaan dan berurutan. Jika ada yang terlewat, struktur data akan mengalami broken link atau kehilangan akses ke sebagian node, yang berpotensi menyebabkan error saat program dijalankan.
4. Pertimbangan Penggunaan DLL: Dibandingkan Singly Linked List, DLL menawarkan fleksibilitas lebih tinggi dalam penelusuran dan penghapusan data. Namun, keunggulan ini dibayar dengan penggunaan memori yang lebih besar karena setiap node menyimpan dua referensi. Oleh karena itu, pemilihan struktur data harus disesuaikan dengan karakteristik masalah yang akan diselesaikan.

Secara keseluruhan, praktikum ini membantu mengubah pemahaman teori struktur data menjadi keterampilan pemrograman yang konkret.

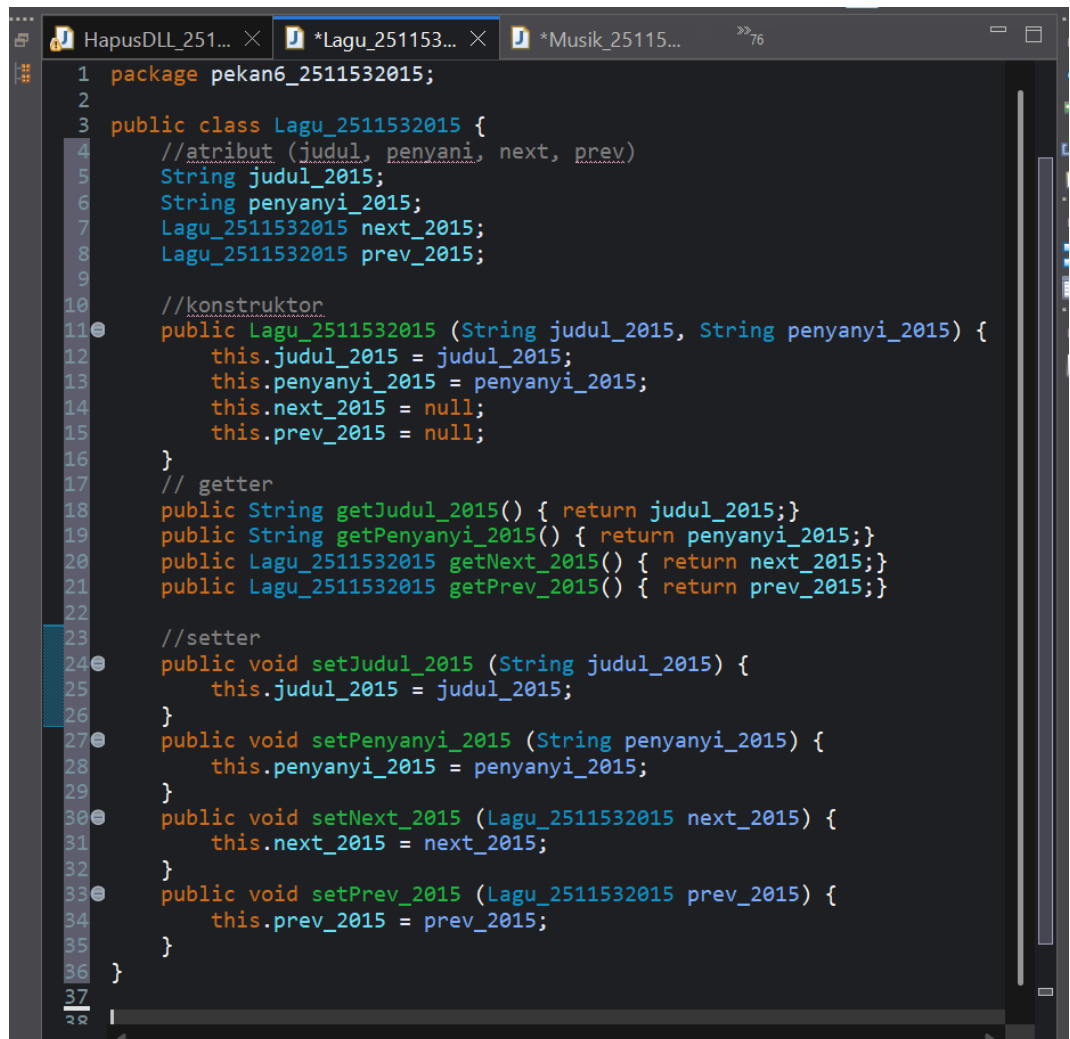
Pengalaman langsung dalam mengelola referensi memori, menangani logika dua arah, dan melakukan debugging pointer menjadi fondasi penting untuk mempelajari struktur data yang lebih kompleks, seperti Stack, Queue, Tree, atau Graph, di modul praktikum selanjutnya.

DAFTAR PUSTAKA

- [1] A. W. e. al., "Penggunaan Algoritma Doubly Linked List Untuk Insertion," *JRIIN: Jurnal Riset Informatika dan Inovasi*, vol. I, no. 12, pp. 1329-1331, 2024.
- [2] Diana, "Linked List: Single, Double, dan Circular Linked List," Telkom University, [Online]. Available: <https://jakarta.telkomuniversity.ac.id/linked-list-single-double-dan-circular-linked-list/>. [Accessed 12 Mey 2026].

LAPORAN TUGAS

1. Kelas Lagu_2511532015

The screenshot shows a Java IDE with three tabs: 'HapusDLL_251...', '*Lagu_251153...', and '*Musik_25115...'. The active tab is '*Lagu_251153...', displaying the code for the 'Lagu_2511532015' class. The code is as follows:

```
1 package pekan6_2511532015;
2
3 public class Lagu_2511532015 {
4     //atribut (judul, penyanyi, next, prev)
5     String judul_2015;
6     String penyanyi_2015;
7     Lagu_2511532015 next_2015;
8     Lagu_2511532015 prev_2015;
9
10    //konstruktor
11    public Lagu_2511532015 (String judul_2015, String penyanyi_2015) {
12        this.judul_2015 = judul_2015;
13        this.penyanyi_2015 = penyanyi_2015;
14        this.next_2015 = null;
15        this.prev_2015 = null;
16    }
17    // getter
18    public String getJudul_2015() { return judul_2015;}
19    public String getPenyanyi_2015() { return penyanyi_2015;}
20    public Lagu_2511532015 getNext_2015() { return next_2015;}
21    public Lagu_2511532015 getPrev_2015() { return prev_2015;}
22
23    //setter
24    public void setJudul_2015 (String judul_2015) {
25        this.judul_2015 = judul_2015;
26    }
27    public void setPenyanyi_2015 (String penyanyi_2015) {
28        this.penyanyi_2015 = penyanyi_2015;
29    }
30    public void setNext_2015 (Lagu_2511532015 next_2015) {
31        this.next_2015 = next_2015;
32    }
33    public void setPrev_2015 (Lagu_2511532015 prev_2015) {
34        this.prev_2015 = prev_2015;
35    }
36 }
37
38
```

Kelas ini berfungsi sebagai pondasi atau blueprint yang bisa dipanggil di kelas lain seperti kelas Musik_2511532015

2. Kelas Musik_2511532015

Kelas Musik_2511532015 berfungsi sebagai **pengelola utama** (controller) untuk sistem playlist berbasis Doubly Linked List (DLL). Jika kelas Lagu_2511532015 adalah "gerbong" yang menyimpan data, maka kelas Musik adalah "masinis" yang mengatur bagaimana gerbong-gerbong tersebut disambung, diputus, dan ditampilkan.

Kelas ini bekerja secara *static*, artinya semua method dapat dipanggil langsung tanpa membuat objek Musik berulang kali. Setiap operasi

manipulasi data (tambah/hapus) akan mengembalikan nilai head terbaru, sehingga struktur playlist selalu *up-to-date*. Integrasi dengan variabel tail membuat operasi tampil mundur menjadi lebih efisien karena tidak perlu menelusuri seluruh daftar dari awal.

```
package pekan6_2511532015;

import java.util.Scanner;

public class Musik_2511532015 {
    private static Lagu_2511532015 tail_2015 = null;
    //method menambahkan lagu baru di akhir playlist (insertEnd)
    public static Lagu_2511532015 tambahLagu_2015
    (Lagu_2511532015 head_2015, Lagu_2511532015 laguBaru) {
        //jika list kosong, lagu baru langsung jadi head
        if (head_2015 == null) {
            head_2015 = laguBaru;
            tail_2015 = laguBaru;
        } else { //jika list ada isinya
            Lagu_2511532015 curr_2015 = head_2015;
            while (curr_2015.next_2015 != null) {
                System.out.println ("Judul: "+
curr_2015.getJudul_2015());
                System.out.println ("Penyanyi: "+
curr_2015.getPenyanyi_2015());
                System.out.println ("Lagu berhasil
ditambahkan");
                curr_2015 = curr_2015.next_2015;
            }
            curr_2015.next_2015 = laguBaru; //node terakhir next
ke lagu baru
            laguBaru.prev_2015 = curr_2015; //lagu baru prev ke
node terakhir
            laguBaru.next_2015 = null; //next lagu baru null
karena paling akhir
            tail_2015 = laguBaru;
        }
        return head_2015;
    }

    //method menghapus lagu pertama (head) menggunakan delHead
    public static Lagu_2511532015 hapusLaguAwal_2015
    (Lagu_2511532015 head_2015) {
        if (head_2015 == null) {
            tail_2015 = null;
            return null;
        }
        head_2015 = head_2015.next_2015;
        if (head_2015 != null) {
            head_2015.prev_2015 = null;
        }
        return head_2015;
    }
}
```

```

//method menampilkan semua lagu dari awal ke akhir
(forwardTraversal)
public static void tampilMaju_2015 ( Lagu_2511532015
head_2015) {
    Lagu_2511532015 curr_2015 = head_2015;
    //cek kosong ga
    if (head_2015 == null) {
        System.out.println ("Playlist masih kosong");
        return;
    }
    //tampil maju (loop)
    while (curr_2015 != null) {
        System.out.print (curr_2015.getJudul_2015()+" - " +
curr_2015.getPenyanyi_2015() + " <-> ");
        curr_2015 = curr_2015.next_2015;
    }
    System.out.println ();
}
//method menampilkan semua lagu dari awal ke akhir
(backwardTraversal)
public static void tampilMundur_2015 (Lagu_2511532015
tail_2015) {
    //mualai dari akhir
    if (tail_2015 == null) {
        System.out.println ("Playlist masih kosong");
        return;
    }
    Lagu_2511532015 curr_2015 = tail_2015;
    while (curr_2015 != null) {
        System.out.print (curr_2015.getJudul_2015()+" - " +
curr_2015.getPenyanyi_2015() + " <-> ");
        curr_2015 = curr_2015.prev_2015;
    }
    System.out.println ();
}
//method mencari lagu berdasarkan judul (tidak
case_sensitive)
public static Lagu_2511532015 cariLagu_2015 (Lagu_2511532015
head_2015, String judulCari_2015) {
    Lagu_2511532015 curr_2015 = head_2015;
    if (head_2015 == null) {
        System.out.println ("Playlist masih kosong");
        return null;
    }
    //traversal maju dari head
    while (curr_2015 != null) {
        //bandingkan
        if
(curr_2015.getJudul_2015().equalsIgnoreCase(judulCari_2015)){
            return curr_2015;
        }
        curr_2015 = curr_2015.next_2015;
    }
    return null;
}
//menampilkan menu
public void tampilkanMenu_2015() {

```

```

        System.out.println ("\n=== Playlist Musik NIM: 2511532015
        ===\n");
        System.out.println ("1. Tambah lagu");
        System.out.println ("2. Hapus Lagu Pertama");
        System.out.println ("3. Lihat Playlist (maju)");
        System.out.println ("4. Lihat Playlist (mundur)");
        System.out.println ("5. Cari Lagu");
        System.out.println ("6. Keluar");
    }
    //main method
    public static void main(String[] args) {
        Scanner input = new Scanner (System.in);
        Lagu_2511532015 head_2015 = null; //nyimpan variable head
        playlist
        int pilih;
        do {
            Musik_2511532015 musik = new Musik_2511532015();
            musik.tampilkanMenu_2015();
            System.out.print("Pilih menu (1-6): ");
            pilih = input.nextInt();
            input.nextLine();
            System.out.println();

            switch (pilih) {
                case 1:
                    System.out.print ("Judul: ");
                    String judul_2015 = input.nextLine();
                    System.out.print ("Penyanyi: ");
                    String penyanyi_2015 = input.nextLine();

                    Lagu_2511532015 laguBaru = new
                    Lagu_2511532015 (judul_2015, penyanyi_2015);
                    head_2015 = tambahLagu_2015(head_2015,
                    laguBaru);
                    tail_2015 = laguBaru;
                    System.out.println(" Lagu berhasil
                    ditambahkan\n");
                    break;

                case 2:
                    head_2015 = hapusLaguAwal_2015 (head_2015);
                    System.out.println ("Lagu pertama dihapus");
                    break;

                case 3:
                    System.out.print ("Playlist (maju): ");
                    tampilMaju_2015(head_2015);
                    System.out.println ();
                    break;

                case 4:
                    System.out.print("Playlist (Mundur): ");
                    tampilMundur_2015(tail_2015);
                    System.out.println();
                    break;

                case 5:

```

```

        System.out.print ("Cari judul lagu: ");
        String cari_2015 = input.nextLine();
        Lagu_2511532015 hasil = cariLagu_2015
(head_2015, cari_2015);
        if (hasil != null) {
            System.out.println ("Lagu ditemukan: "+
hasil.getJudul_2015()+ " - "+ hasil.getPenyanyi_2015()+"\n");
        } else {
            System.out.println ("Lagu tidak ada di
playlist");
        } break;

        case 6:
            System.out.println ("Keluar dari program,
terima kasih");
            break;

        default:
            System.out.println ("Pilihan tidak valid");
        }
    }
    while (pilih != 6);
    input.close();
}
}

```

```
Musik_2511532015 [Java Application] C:\Users\USER\p2\pool\p

=== Playlist Musik NIM: 2511532015 ===

1. Tambah lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (maju)
4. Lihat Playlist (mundur)
5. Cari Lagu
6. Keluar
Pilih menu (1-6): 1

Judul: Rosalia
Penyanyi: Mio Cristo Piange Diamanti
Lagu berhasil ditambahkan

=== Playlist Musik NIM: 2511532015 ===

1. Tambah lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (maju)
4. Lihat Playlist (mundur)
5. Cari Lagu
6. Keluar
Pilih menu (1-6): 1

Judul: All I ask
Penyanyi: Adele
Lagu berhasil ditambahkan
```

Ketika pengguna menginputkan atau memilih pilihan 1, program akan meminta pengguna untuk menginputkan judul dan penyanyi dari lagu yang akan ditambahkan, ketika selesai program akan mengeluarkan output “Lagu berhasil ditambahkan”.

```
=== Playlist Musik NIM: 2511532015 ===

1. Tambah lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (maju)
4. Lihat Playlist (mundur)
5. Cari Lagu
6. Keluar
Pilih menu (1-6): 3

Playlist (maju): Rosalia - Mio Cristo Piange Diamanti <-> All I ask - Adele <->

=== Playlist Musik NIM: 2511532015 ===

1. Tambah lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (maju)
4. Lihat Playlist (mundur)
5. Cari Lagu
6. Keluar
Pilih menu (1-6): 4

Playlist (Mundur): All I ask - Adele <-> Rosalia - Mio Cristo Piange Diamanti <->
```

Setelah lagu ditambahkan pengguna bisa mengecek atau melihat playlist maju dan mundur dengan memilih pilihan 3 dan 4. Dimana jika playlist

maju, akan muncul output daftar playlist yang diurutkan dari depan ke belakang, dan jika mundur maka sebaliknya dari belakang ke depan.

```
=== Playlist Musik NIM: 2511532015 ===  
1. Tambah lagu  
2. Hapus Lagu Pertama  
3. Lihat Playlist (maju)  
4. Lihat Playlist (mundur)  
5. Cari Lagu  
6. Keluar  
Pilih menu (1-6): 2  
  
Lagu pertama dihapus  
  
=== Playlist Musik NIM: 2511532015 ===  
1. Tambah lagu  
2. Hapus Lagu Pertama  
3. Lihat Playlist (maju)  
4. Lihat Playlist (mundur)  
5. Cari Lagu  
6. Keluar  
Pilih menu (1-6): 3  
  
Playlist (maju): All I ask - Adele <->
```

Pengguna juga bisa menghapus lagu paling teratas dengan memilih pilihan ke 2, dimana akan bekerja method delHead.

```
=== Playlist Musik NIM: 2511532015 ===  
1. Tambah lagu  
2. Hapus Lagu Pertama  
3. Lihat Playlist (maju)  
4. Lihat Playlist (mundur)  
5. Cari Lagu  
6. Keluar  
Pilih menu (1-6): 5  
  
Cari judul lagu: all i ask  
Lagu ditemukan: All I ask - Adele
```

Selanjutnya jika pengguna memilih pilihan ke 5, atau cari lagu berdasarkan judul, program akan meminta pengguna meng inputkan judul lagu yang ingin dicari, jika lagu tersebut ada di playlist program akan memunculkan data lagu, dan jika tidak program akan menampilkan, lagu tidak ada di playlist.

```
=== Playlist Musik NIM: 2511532015 ===
```

1. Tambah lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (maju)
4. Lihat Playlist (mundur)
5. Cari Lagu
6. Keluar

```
Pilih menu (1-6): 6
```

```
Keluar dari program, terima kasih
```

Terakhir, pilihan ke 6 untuk keluar dari program.